Counselling Code : VBIT

VIGNANA BHARATHI Institute of Technology ®

(A UGC Autonomous Institution, Approved by AICTE, Accredited by NBA & NAAC-A Grade, Affiliated to JNTUH)

Department of Computer Science and Engineering

Minor Project Abstract Review

Title: Object Detection using Python and YOLO

Project Guide

Dr.Akula Sreenivasulu Sir
Professor

Team members

P. Varshith - 22P61A05J1

Naga Sai Karthikeya.S - 22P61A05G4

Sree Vardhan - 22P61A05H6

OUTLINE

- Abstract
- Introduction
- Existing system
- Proposed system
- Advantages of Proposed System
- Future scopes of Project
- Requirements & Specification
- Project Time Plan
- Literature Survey
- UML
- Conclusion
- References

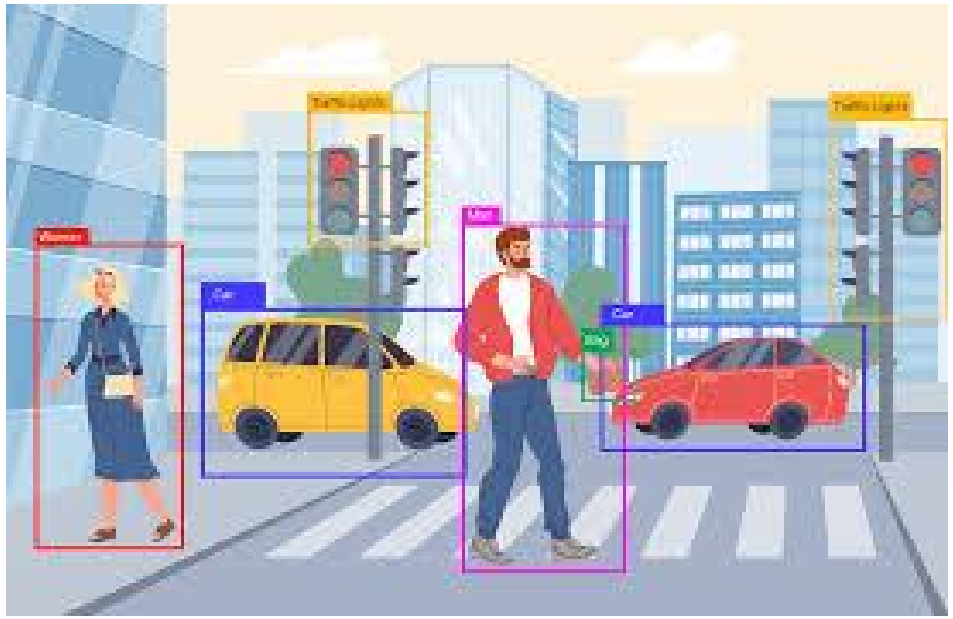
ABSTRACT

- Object detection is a fundamental task in artificial intelligence (AI) and computer vision, playing a crucial role in applications such as surveillance, autonomous driving, and robotics.

- Object detection is a critical application in computer vision, enabling machines to identify and locate objects in images or videos. This project focuses on real-time object detection using the YOLO (You Only Look Once) model integrated with OpenCV. YOLO is a deep learning-based algorithm that performs fast and accurate detection, making it ideal for real-time applications. The project utilizes pre-trained YOLO models to detect multiple objects in images and video streams, highlighting their positions with bounding boxes.

INTRODUCTION

- **Object Detection** is a type of computer vision task where the goal is to identify and locate multiple objects within an image or video.Object detection classifies objects and their position by using bouding boxes.
- **Bounding Boxes** are rectangular boxes drawn around detected objects in an image or video to indicate their position and size. They are a core component of object detection models, helping the model localize and classify objects.
- **YOLO(You Only Look Once)** is a deep learning-based algorithm which performs fast and accurate detection, making it ideal for real-time applications



EXISTING SYSTEM

- Traditional object detection systems rely on classic computer vision techniques or older deep learning models like R-CNN(Computer Neural Networks).These systems involve multiple stages, making them computationally expensive and slow.

- They also require significant processing power, which makes real time object detection difficult, and in resource-limited environments, these methods are inefficient due to their high latency.

- Additionally, many systems also suffer with detecting small or overlapping objects in crowded scenes.

- Traditional models such as HOG(Histogram of Oriented Gradients, another object detection model), require extensive training data and high performance hardware to work properly.

PROPOSED SYSTEM

- The proposed system uses YOLO(You-Only-Look-Once) algorithm for real time detection, which significantly improves speed and efficiency.

- Unlike Traditional models, this algorithm processes the image in a single forward pass, enabling real time detection with high accuracy.

- It divides the input image into a grid and predicts bounding boxes and class probabilities simultaneously, reducing computation time.

- This model is integrated with OpenCV(Open Computer Vision) which helps the model to view the objects in the environment. This model ensures low latency and high performance.

ADVANTAGES OF PROPOSED SYSTEM OVER EXISTING SYSTEM

- **Speed:** YOLO model is fast as it detects objects in a single pass.
- **Real-time performance:** Optimised for real time detection
- **Detection accuracy:** Comparable accuracy with much faster processing speed
- **Small object detection:** Improved detection for small objects
- **Overlapping object handling:** Handles overlapping objects efficiently
- **Use cases:** Ideal for real-time applications such as surveillance, autonomous driving etc.

FUTURE SCOPES OF OBJECT DETECTION MODEL

Security and Surveillance

- Intruder detection: Can be used to detect unauthorized individuals in restricted areas.
- Crowd Monitoring: To detect suspicious activities in large gatherings.

Retail and Inventory

- Shelf Monitoring: Identify empty shelves and restock them efficiently.
- Theft Preventions: Detect unusual activites such as shoplifting in retail stores.

Traffic Management

- Illegal Parking Detection: Identify vehicles which are parked in no parking zones.
- Traffic Congestion Monitoring: Detect vehicle density and optimize signal timings.

SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS(Minimum)

- **Processor** : Intel Core i5
- **RAM** : 8 GB RAM
- **GPU** : INTEL IRIS XE or More
- **Storage** : 50 GB HDD

SOFTWARE REQUIREMENTS

- **Operating System** : Windows or MacOS
- **Programming Language** : Python
- **Frameworks** : YOLO, OpenCV, PyTorch or Tensor flow

PROJECT TIME PLAN

SNO	DURATION	TASK
1.	10.02.2025 – 15.02.2025	Submission of abstract
2.	24.02.2025 – 01.03.2025	Literature survey
3.	24.03.2025 – 29.03.2025	Project Design
4.	14.04.2025 – 19.04.2025	Implementation
5.	28.04.2025 – 03.05.2025	Testing & Validation
6.	---	--
7.	15.04.2025	Submission of Publication document
8.	25.04.2025	Submission of Final Project Document

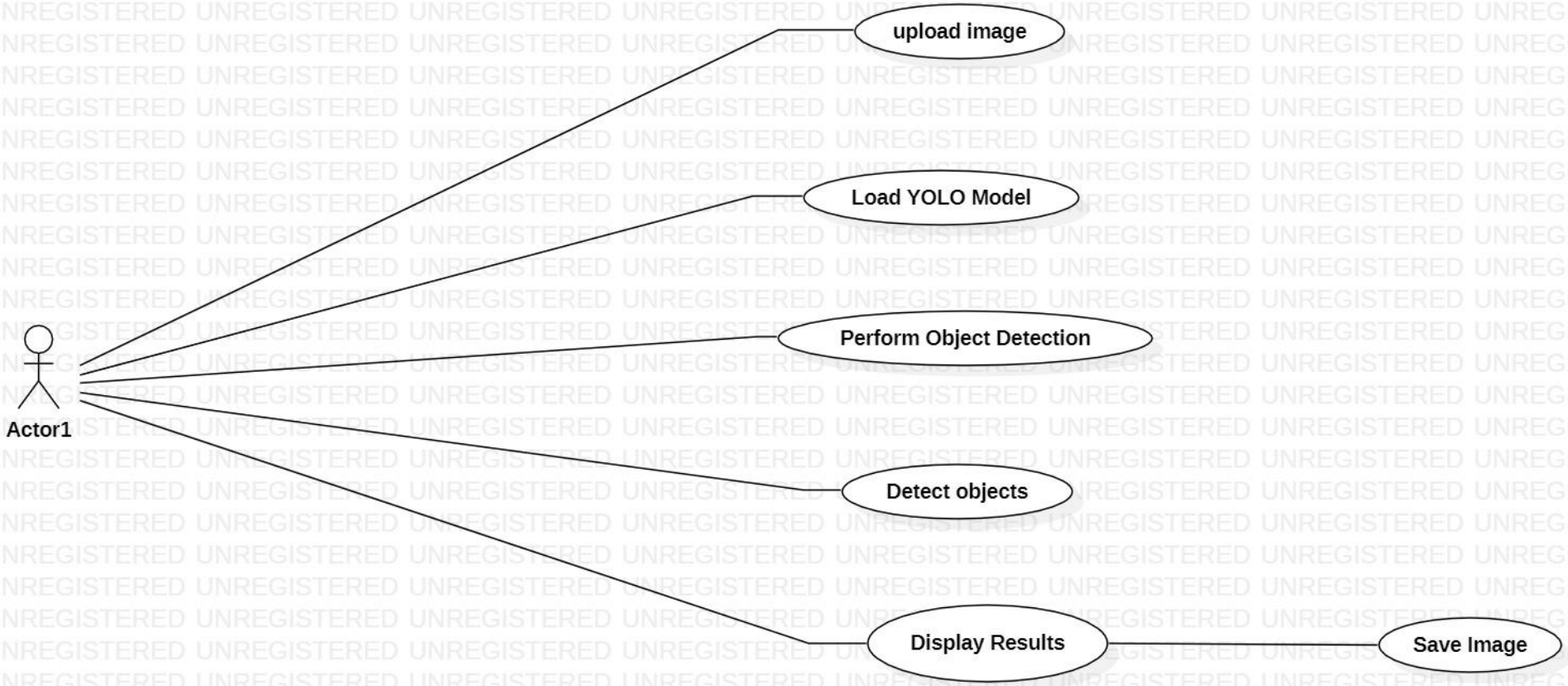
LITERATURE SURVEY

TITLE	AUTHOR	YEAR	ADVANTAGES	LIMITATIONS
1)**Object Detection (using R-CNN)**	Yali Amit, Ross Girshik	2021	Can detect objects of several different instances of classes such as people, cars, or any other objects. Each detection is reported with some information, such as their location using bounding boxes.	Can only detect if limited number of objects (less than 10) are present in the image. Speed is very slow, and accuracy is not high.
2)**Scalable High Quality Object Detection**	Scotty reed, Sergey Loffey, Dumitru Erhan	2015	Improved recognition, accurate(better accuracy than CNN)	Not memory and resource efficient Sometimes might be slow when multiple objects are present

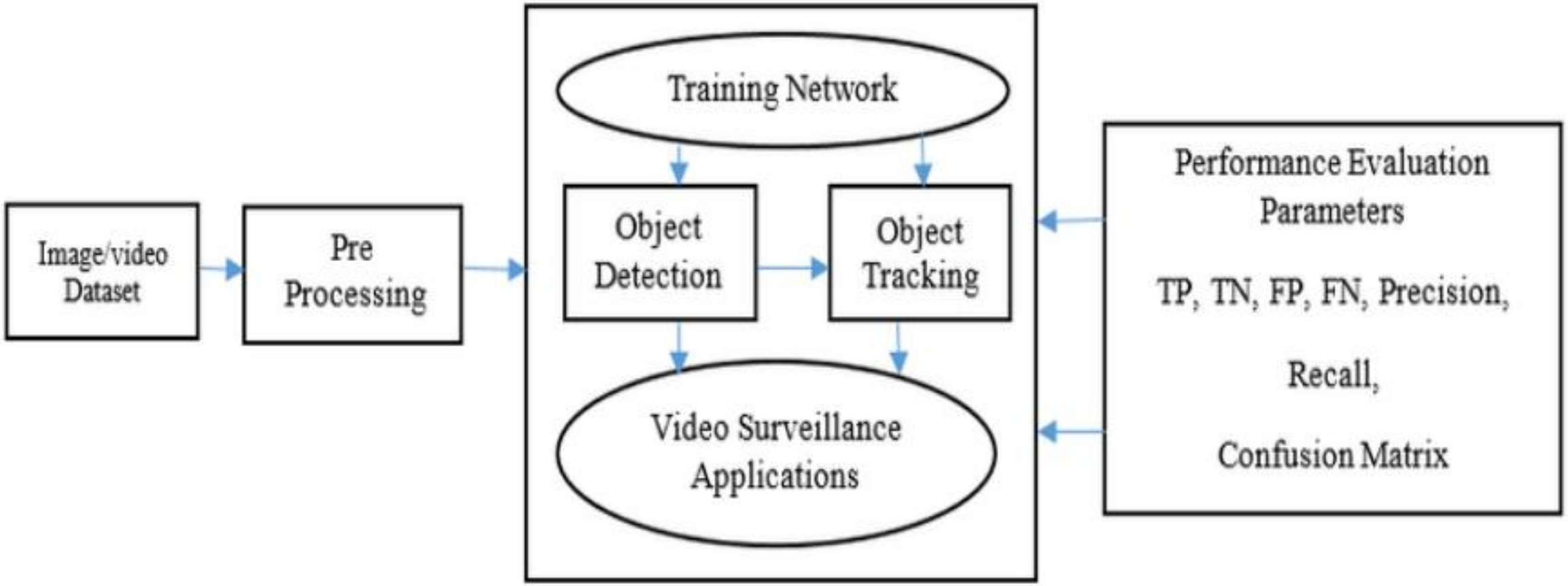
3)Underwater object detection	Muwei Jian, Nan Yang,	2024	The rapidly growing exploitation and utilization of marine resources by humans has sparked considerable interest in underwater object detection tasks. Targets captured in underwater environments differ significantly from those captured in	These adverse factors pose a host of challenges, such as high intensity noise, texture distortion, uneven illumination, low contrast, and limited visibility in underwater images
4)**Finding exoplanets using object detection**	S. R. Mani Sekhar, C. Tejas, V. S. Navneet Kanna & Aasees Kaur	2023	Object detection algorithms can process large datasets quickly (e.g., from telescopes like Kepler, TESS, or future surveys), far faster than manual analysis.	Object detection methods heavily depend on data quality. Noisy, incomplete, or biased datasets can severely reduce performance.

5)***Object Detection Using Haar Cascades***	Viola and Jones	2001	Can detect objects quickly, even on simple computers, Good for real-time applications like face detection, Uses easy mathematical features, making it lightweight, Good at finding objects like front-facing faces.	Hard to detect very small or far away objects, Struggles if the background is messy, Needs many positive and negative examples, Fixed object types Works best for the type of object it was trained for (like faces).

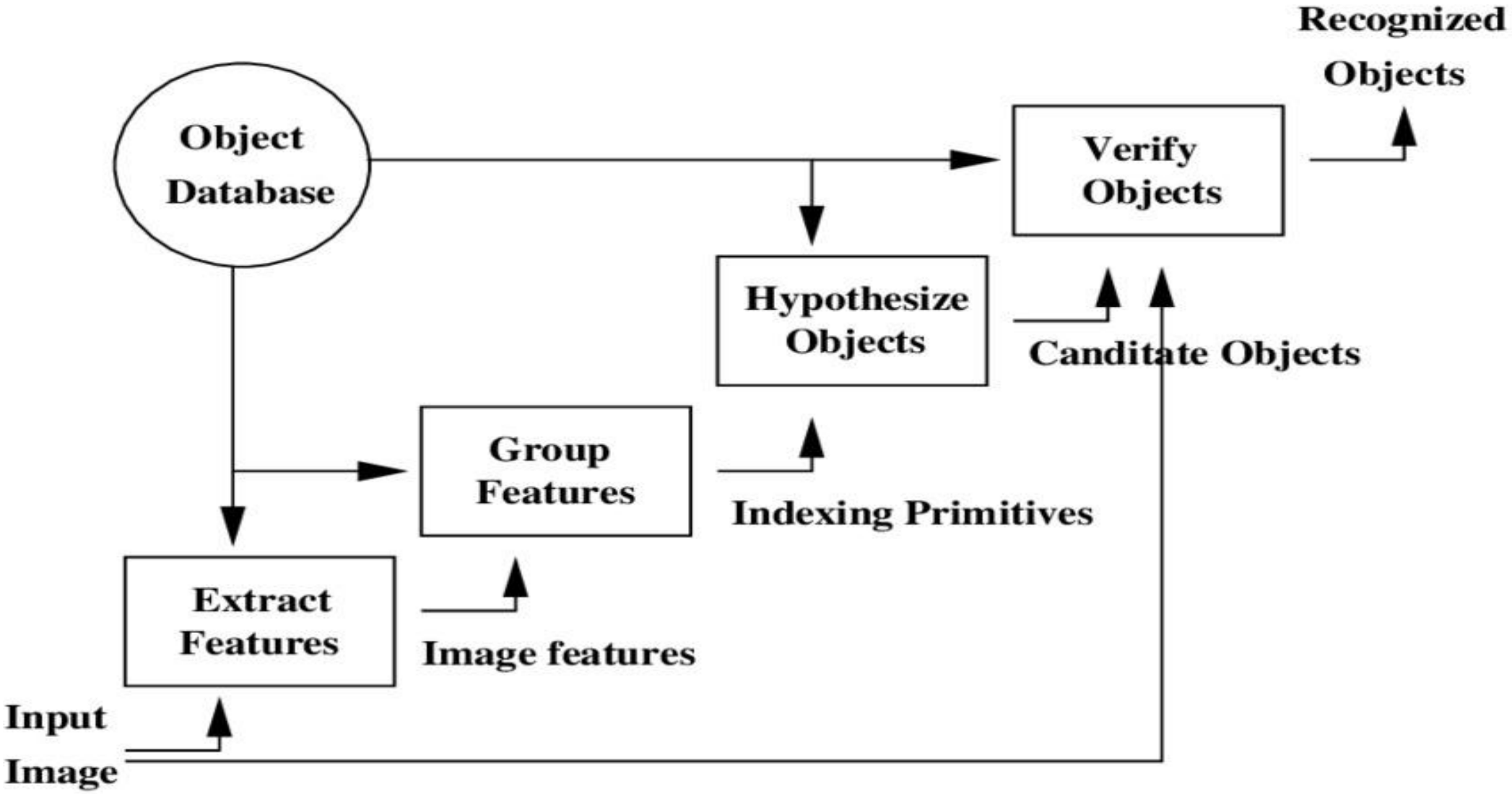
USE CASE DIAGRAM



Flow Chart


Image/video Dataset
Pre Processing
Training Network
Object Detection
Object Tracking
Video Surveillance Applications
Performance Evaluation Parameters
TP, TN, FP, FN, Precision,
Recall,
Confusion Matrix

Component Diagram



CODE

```
!pip install ultralytics
```

[1] Python

```
Collecting ultralytics
  Downloading ultralytics-8.3.123-py3-none-any.whl.metadata (37 kB)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (4.11.0.86)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (11.2.1)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (6.0.2)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (2.32.3)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (1.15.2)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (2.6.0+cu124)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (0.21.0+cu124)
Requirement already satisfied: tqdm>=4.64.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (4.67.1)
Requirement already satisfied: psutil in /usr/local/lib/python3.11/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: py-cpuinfo in /usr/local/lib/python3.11/dist-packages (from ultralytics) (9.0.0)
Requirement already satisfied: pandas>=1.1.4 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (2.2.2)
Requirement already satisfied: seaborn>=0.11.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (0.13.2)
Collecting ultralytics-thop>=2.0.0 (from ultralytics)
  Downloading ultralytics_thop-2.0.14-py3-none-any.whl.metadata (9.4 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.4.8)
```

```
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3.0->ultralytics) (24.2)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0.post0)
...
    Found existing installation: nvidia-cusolver-cu12 11.6.3.83
    Uninstalling nvidia-cusolver-cu12-11.6.3.83:
      Successfully uninstalled nvidia-cusolver-cu12-11.6.3.83
Successfully installed nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-cu12-12.4.127 nv
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

[16]

```python
from google.colab import files
from ultralytics import YOLO
from PIL import Image
import io
import matplotlib.pyplot as plt
```

Python

[17]

```python
model = YOLO("yolov8n.pt")
```

Python

```python
uploaded=files.upload()
```

[21] Python

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving sampleimg2.jpeg to sampleimg2.jpeg
```

```python
if uploaded:
    filename = list(uploaded.keys())[0]
    print(f"Uploaded file: {filename}")

    # STEP 5: Load and run YOLO on the image
    model = YOLO("yolov8n.pt")  # Load pre-trained YOLOv8 model
    results = model(filename)  # Perform detection

    # STEP 6: Show detection output
    results[0].show()  # Inline display in Colab
else:
    print("File not uploaded")
```

[22] Python

OUTPUT:

```
Uploaded file: sampleimg2.jpeg

image 1/1 /content/sampleimg2.jpeg: 576x640 1 bicycle, 1 car, 1 truck, 1 dog, 219.2ms
Speed: 5.5ms preprocess, 219.2ms inference, 1.3ms postprocess per image at shape (1, 3, 576, 640)
```

CONCLUSION

YOLO (You Only Look Once) has revolutionized object detection by providing a fast, efficient, and accurate deep-learning approach. Unlike traditional models which involve multiple processing stages, YOLO performs detection in a single pass, making it highly suitable for real-time applications. This makes it ideal for fields such as autonomous driving, security surveillance, industrial automation, and medical imaging.

REFERENCES

- 1)https://link.springer.com/referenceworkentry/10.1007/978-3-030-63416-2_660
- 2)https://arxiv.org/abs/1412.1441
- 3)https://link.springer.com/article/10.1007/s44295-024-00023-6#Abs1
- 4)https://link.springer.com/article/10.1007/s10509-023-04232-z
- 5)https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf